

ChickpeaOmicsR

This package provides gene expression data for chickpea under different stress conditions. Several Data MayBe Loaded to the Environment. Gene Expression counts are normalized. Alsamman M. Alsamman, aalsamman100[at and not .]gmail[now .]com.

ChickpeaOmicsR offers a comprehensive toolkit for chickpea breeders, enabling them to:

- Identify key genes involved in important traits like flowering, salinity tolerance, and disease resistance.
- Visualize gene expression patterns and protein interaction networks to understand complex molecular mechanisms.
- Integrate multi-omics data, including GWAS, to discover candidate genes for breeding programs.
- Conduct gene enrichment analysis to explore biological processes and pathways relevant to chickpea improvement.
- Access preloaded datasets for quick analysis and visualization of chickpea omics data.
- Make informed breeding decisions based on genotype-phenotype associations and gene expression profiles.
- Enhance collaboration and knowledge sharing among chickpea researchers through standardized data analysis tools.

By leveraging **ChickpeaOmicsR**, breeders can make more informed decisions in chickpea breeding programs, ultimately improving crop yield and resilience. Each function supports common workflows in chickpea research, helping breeders and biologists analyze omics data efficiently and gain insights into gene expression, protein interactions, and genetic associations.

Installation

This packages will installed automatically when you install ChickpeaOmicsR. But it can be installed separately as well to make things easier.

```
library(devtools)
```

```
## Loading required package: usethis
```

```
#install.packages("devtools")
```

```
library(igraph)
```

```
##
```

```
## Attaching package: 'igraph'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      decompose, spectrum
```

```
## The following object is masked from 'package:base':
```

```
##
```

```
##      union
```

```
#install.packages("igraph")
```

```
library(pheatmap)
```

```
#install.packages("pheatmap")
```

```
library(ggplot2)
```

```
#install.packages("ggplot2")
```

```
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:igraph':
```

```
##
##   as_data_frame, groups, union
## The following objects are masked from 'package:stats':
##
##   filter, lag
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
#install.packages("dplyr")
```

You can install the released version of ChickpeaOmicsR from CRAN with:

```
library(ChickpeaOmicsR)
```

```
## Welcome to ChickpeaOmicsR!
## [1] "This package provides gene expression data for chickpea under different stress conditions."
## [1] "Several Data MayBe Loaded to the Environment."
## [1] "Gene Expression counts are normalized"
## [1] "Alsamman M. Alsamman, aalsamman100[at and not .]gmail[now .]com"
#devtools::install_github("AlsammanAlsamman/ChickpeaOmicsR")
```

Datasets

ChickpeaOmicsR package provides several datasets that can be used to perform various analyses and visualizations. The datasets are as follows:

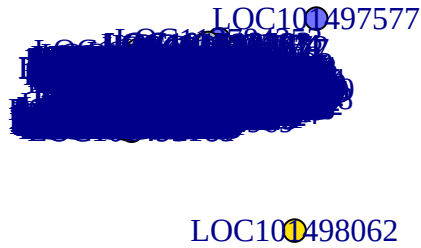
1. **CaExpressionCounts**: Gene expression counts for chickpea. Generated from RNA-seq data of Gene expression counts for 28891 gene and 197 samples.
2. **CaExpressionMetadata**: Metadata associated with gene expression studies.
3. **CaGenomeAnnotation**: Genome annotations of chickpea genes.
4. **CaProteinEnrichment**: Protein enrichment analysis of all Chickpea genes.
5. **CaProteinSequenceInfo**: Protein sequence information.
6. **CaProteinInteractionNetwork**: Protein-protein interaction data.
7. **CaExpressionSignificance**: Data on significant gene expression changes.
8. **CaGWAS**: Genome-wide association study data for chickpea.

Example usage for studing genes associated with flowering

This feature helps chickpea breeders identify key genes related to traits like flowering or stress responses.

```
#Retrieve gene expression data by Gene Ontology (GO) terms and plot heatmaps to visualize expression pa
# Retrieve genes associated with specific GO terms or keywords
genes <- get_geneids_bygotermkeywords(c("Flowering"))
# Extract log fold-change values for gene expression
geneExp <- get_gene_logfc_bycategory(unlist(genes), pvalue = 0.01)
## [1] "some genes may have been removed due to pvalue"
# Plot a heatmap of the gene expression
plot_gene_expression_heatmap(geneExp, plot_height = 10, plot_width = 10, file = "gene_expression_heatmap.pdf")
## [1] "Heatmap saved to gene_expression_heatmap.pdf"
```


Protein Interaction Network



Edge lengths represent score

```
## pdf
## 2
```

This feature is particularly useful for chickpea breeders who want to understand how protein interactions influence important traits.

GO Term and GWAS Integration

Combine gene expression data with GWAS results for a more comprehensive view of genotype-phenotype associations.

```
# Generate a gene-phenotype matrix based on GO term and GWAS data
gene_pheno_matrix <- gene_pheno_GWASmatrix_byGOKeyword("Flowering", pvalue = 0.05)
```

```
## Aggregation function missing: defaulting to length
```

```
# Plot the gene-phenotype matrix
```

```
plot_gene_pheno_GWASmatrix_byGOKeyword(gene_pheno_matrix, outfolder = "heatmaps", plotwidth = 5, plotheight = 5)
```

```
## Using geneid as id variables
```

This allows breeders to associate specific gene functions with phenotypic traits, speeding up marker discovery and breeding decisions.

Functions Overview

Suppose you're a chickpea researcher interested in how different stress factors (like salinity) affect gene expression. You can use the `get_study_data("Salinity")` function to extract the gene expression counts and metadata specifically for salinity studies, allowing you to focus on this area of interest.

```
# Example usage
salinity_data <- get_study_data(study = "Salinity")
head(salinity_data$counts)
```

```
##          SRR1066056.2 SRR1066060.2 SRR10331638 SRR10331639 SRR10331640
## LOC101498889          1          71922          NA          NA          NA
## LOC101512000          3          70798           2           0           0
## LOC101499824        57297           0          48         237         107
## LOC101507652          5         34498          NA          NA          NA
## LOC101499528         26         53341          10           6           7
## LOC101514962          0         15931          NA          NA          NA
##          SRR10331641 SRR10331642 SRR10331643 SRR10331644 SRR10331645
## LOC101498889          NA          NA          NA          NA          NA
## LOC101512000          11           1           0           0           0
## LOC101499824        3404         679          32         381        1116
## LOC101507652          NA          NA          NA          NA          NA
## LOC101499528          94           8           7         29           7
## LOC101514962          NA          NA          NA          NA          NA
##          SRR1066058 SRR1066062 SRR2081143 SRR2081144 SRR2081145 SRR2081146
## LOC101498889          0         85260           0           0           0           0
## LOC101512000          2         75849           2           5          12           1
## LOC101499824       15296           3          48         285        1208       4559
## LOC101507652          2        16355          NA          NA          NA          NA
## LOC101499528         13        63134           1           0           0           0
## LOC101514962          0        23906          NA          NA          NA          NA
##          SRR2081147 SRR2081148 SRR2081149 SRR2081150 SRR2081151 SRR2081152
## LOC101498889          1           0           0           0           2           1
## LOC101512000          3           0           1           0           0           3
## LOC101499824        310           4         736         319         335         94
## LOC101507652          NA          NA          NA          NA          NA          NA
## LOC101499528          2           0           1           0           0           0
## LOC101514962          NA          NA          NA          NA          NA          NA
##          SRR2081153 SRR2081154 SRR2081155 SRR2081156 SRR2081157 SRR2081158
## LOC101498889          0           1           0           0           8           5
## LOC101512000          0           0           0           3          15          16
## LOC101499824        333         538        82178        11736        3341        5632
## LOC101507652          NA          NA          NA          NA          NA          NA
## LOC101499528          0           1           1          11         23         23
## LOC101514962          NA          NA          NA          NA          NA          NA
```

2. **Retrieve Data for Specific Samples:** If you're conducting a follow-up analysis on specific samples collected from your experiments, you can use the `get_samples_data()` function to subset gene expression data and metadata for only those samples. For example, extracting data from SRR10331638 to SRR10331645:

```
# Example usage
samples_data <- get_samples_data(c("SRR10331638", "SRR10331639", "SRR10331640"))
head(samples_data$counts)
```

```
##          SRR10331638 SRR10331639 SRR10331640
## LOC101498889          NA          NA          NA
## LOC101512000          2           0           0
## LOC101499824         48         237         107
## LOC101507652          NA          NA          NA
## LOC101499528         10           6           7
## LOC101514962          NA          NA          NA
```

3. **Obtain Metadata for a Group of Interest:** To get metadata based on specific study groups, experiments, or samples, you can use `get_metadata()`. For instance, you may want metadata about an experiment titled "PRJNA232700":

```
# Example usage
experiment_metadata <- get_metadata(experiment = "PRJNA232700")
head(experiment_metadata)
```

```
## [1] group      experiment study
## <0 rows> (or 0-length row.names)
```

4. **Analyze GWAS Results for Specific Genes:** If you're interested in a GWAS analysis to identify associations between gene variants and traits like drought tolerance, the `get_gwas_data()` function allows you to filter the data by gene and p-value. This could help you find significant associations for your genes of interest.

```
# Example usage
gwas_data <- get_gwas_data(c("GRX3", "HIS2A"), pvalue = 0.01)
head(gwas_data)
```

```
## [1] chr      ps      allele1    allele0    af
## [6] beta      se      pvalue     phenotype  organization
## [11] yearMonth rs      Chr      geneid     pangenomegene
## <0 rows> (or 0-length row.names)
```

5. **Fetch Gene Annotations:** To explore gene annotations, `get_gene_annotation_bygeneid()` retrieves functional information about the genes you're interested in. This function helps integrate gene functional context into your study:

```
# Example usage
gene_annotation <- get_gene_annotation_bygeneid(c("GRX3", "HIS2A"))
head(gene_annotation)
```

```
##      chr      start      end strand      proteinid stringdbid
## 73 NC_021162.1 23809150 23811422      - 3827.AOA3Q7K8F5 AOA3Q7K8F5
## 74 NC_021162.1 38421374 38422204      + 3827.065759      065759
##
## 73
## 74 Histone H2AX; Variant histone H2A which replaces conventional H2A in a subset of nucleosomes. Nuc
##      geneid
## 73      GRX3
## 74      HIS2A
```

6. **Gene Enrichment Analysis for GO Terms:** For researchers investigating gene ontology (GO) terms related to specific biological processes, the `get_geneids_bygotermkeywords()` function can extract genes associated with keywords like "Flowering" or "Salinity." It allows you to focus on genes involved in relevant pathways:

```
# Example usage
genes_in_flowering <- get_geneids_bygotermkeywords(c("Flowering"))
```

7. **Analyze Protein-Protein Interaction Networks:** If you want to visualize interactions among proteins of interest, you can use `get_protein_interaction_table()` to extract network data and then `plot_protein_interaction_network()` to create a network graph. This helps to identify important interactions among proteins in your study:

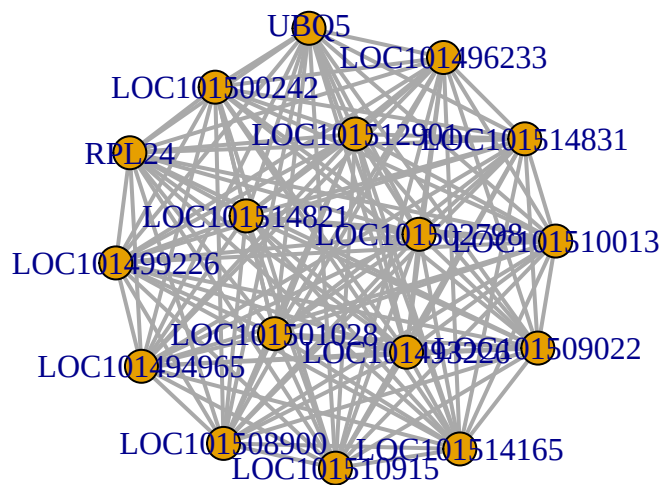
```
# Example usage
genes_query <- c("LOC101514165"
,"LOC101514831"
,"LOC101502798"
,"LOC101509022"
,"LOC101494965")
```

```

,"LOC101510915"
,"LOC101512901"
,"LOC101510013"
,"LOC101501028"
,"LOC101514821"
,"LOC101508900"
,"LOC101493226"
,"LOC101496233"
,"LOC101499226"
,"LOC101500242"
,"RPL24"
,"UBQ5")
ppi_table <- get_protein_interaction_table(genes_query,only_interacting=TRUE)
plot_protein_interaction_network(ppi_table, file = "network.pdf")

```

Protein Interaction Network



Edge lengths represent score

```
## pdf
## 2
```